Schelling-Point Reputation Communities: A Decentralized Guarantee and Arbitration Layer for Agent-to-Agent Commerce

AgentTrust Protocol Team
github.com/Fishman-free/multiagent

August 10, 2026

Abstract

As large language models and tool-use capabilities mature, AI agents are evolving from analytical tools into counterparties capable of autonomously negotiating, signing, transferring funds, and settling. When agents hold funds and act as "digital twins" of humans, anonymous, machine-to-machine commerce raises a new class of trust problems: how can agents establish mutual trust, and how can malicious agents be held accountable after misbehavior? We present a decentralized guarantee and arbitration layer for agent-to-agent commerce built on four production Solidity contracts — an agent registry, a guaranteed escrow, a staked-voting arbitration mechanism, and an on-chain reputation hub — and formalize its core mechanism, the *Schelling-point reputation community*: a set of staked voters who adjudicate trade disputes, whose verdicts drive the release of escrowed funds and the update of on-chain reputation, and whose reputation scores in turn price the guarantees that make future trades safe. We prove that honest voting is an incentive-compatible equilibrium under a two-thirds supermajority rule (an attack by a minority of at most 35% of voters is strategically harmless), that Sybil attacks are unprofitable because adversarial voting power scales linearly in expenditure, and that the mechanism conserves funds exactly. We instantiate the protocol in Solidity (38 tests), validate the game-theoretic claims by Monte-Carlo simulation built on `mesa`, and discuss deployment to a public testnet.

Keywords: AI agents; blockchain; decentralized arbitration; staked voting; Schelling points; reputation; guaranteed escrow; agentic commerce.

1 Introduction

Background: from analysis tool to digital-twin counterparty. Large language models (LLMs) and tool-use capabilities have made AI agents able to plan autonomously, call external tools, and execute multi-step tasks. The decisive shift is that an agent is no longer merely a system that *answers questions*; it can make economically consequential commitments on behalf of its owner — negotiating prices, signing terms, initiating transfers, and settling smart contracts. When an agent holds funds and can complete payments, it effectively constitutes a "digital twin" trading counterparty of its human owner.

This trend has given rise to agentic commerce and agent-to-agent (A2A) finance as research categories [1, 2]. Prior work observes that AI agents are transitioning from analysis tools into counterparties, generating a demand for financial-grade infrastructure for identity, authorization, payment, verification, reputation, and accountability. However, widespread adoption amplifies

three systemic risks: *identity fraud* — anyone can cheaply mass-produce "fake agents"; *mutual-trust deficit* — an anonymous counterparty cannot tell whether the other side is trustworthy; and *accountability vacuum* — after an agent misbehaves there is no clear path to attribute fault, present evidence, and impose sanctions.

Problem statement. We focus on three interconnected questions:

- **Q1 (Trust).** When agents transact on behalf of humans, how can mutual trust be established so that "scammer agents" do not infiltrate?
- **Q2 (Accountability).** When a scammer agent commits fraud, how are fault, evidence, and sanctions determined?
- **Q3 (Incentives).** Which economic and governance mechanisms make the trust infrastructure self-sustaining and scalable?

These questions restate the classic trust problem in a machine-to-machine, anonymity-by-default setting: trust must come not from knowing a counterpart's history but from a verifiable, auditable, and traceable technical infrastructure.

Our approach: a Schelling-point reputation community. We design and implement a decentralized guarantee and arbitration layer for agent-to-agent commerce. The system is organized along the identity–trust–accountability (ITA) architecture (Section 3): a registry issues ERC-721 agent identities bound to real principals; a guaranteed escrow escrows trade funds against guarantor stakes; disputes are adjudicated by a community of staked voters whose majority — by a Schelling-point argument — is incentivized to vote truthfully; and a reputation hub records outcomes on-chain and prices future guarantees. The central novelty is the *closed loop*: verdicts update reputation, reputation prices the guarantee, and the guarantee determines whether low-reputation agents can transact at all (Section 4.4).

The mechanism is instantiated by four production Solidity contracts (Section 5.2) with 38 passing tests, and its game-theoretic claims are validated by Monte-Carlo simulation (Section 5.4).

Contributions.

- **A closed-loop mechanism** connecting adjudication, guarantees, and reputation — the first, to our knowledge, to tie all three into one on-chain cycle (Sections 4, 4.4).
- **A formal game-theoretic analysis** of the staked-voting mechanism: honest voting is an incentive-compatible equilibrium under a two-thirds supermajority rule; a coordinated malicious minority of at most 35% of voters is strategically harmless; Sybil attacks are unprofitable because adversarial voting power scales linearly in expenditure; and the mechanism conserves funds exactly (Section 4.3).
- **A production implementation** — four verified-style Solidity contracts, 38 tests, an end-to-end demo flow, and Docker one-command bootstrapping — together with a mesa-based simulation that empirically validates the theory (Section 5).
- **An honest evaluation** of the MVP's limitations (free-abstention refunds, integer dust, threshold approximation) and the deployment path to a public testnet (Sections 5.4, 5.5).

Organization. Section 2 surveys related work. Section 3 presents the ITA architecture and its mapping to the four contracts. Section 4 formalizes the mechanism and proves its incentive properties. Section 5 describes the implementation and its empirical evaluation. Section 6 concludes.

2 Related Work

2.1 Decentralized identity and verifiable credentials

The W3C has standardized Decentralized Identifiers (DIDs) [3] and Verifiable Credentials (VCs) [4] as mutually independent, composable primitives: a DID is a URI that resolves to a DID document carrying control and verification methods, and a VC is a cryptographically verifiable data model for claims such as qualifications, authorization, and reputation. Implementation layers such as ION [5] anchor DID operations on Bitcoin without a new token. These standards provide the skeleton for issuing identities to agents, but they do not by themselves determine how identities are trusted in commerce, how disputes are adjudicated, or how reputation is priced — the concerns of this paper.

2.2 Soulbound tokens and bound NFTs

Soulbound tokens (EIP-5192 [6]) and account-bound NFTs (EIP-4973 [7]) generalize non-transferability, letting an address hold verifiable, non-tradeable attestations of membership, achievement, or commitment. In our architecture the agent identity (an ERC-721 in AgentRegistry) is bound to a real principal at registration and cannot be detached from the responsibility it carries — the accountability analogue of the soulbound idea.

2.3 Autonomous accounts and payments

Account abstraction (ERC-4337 [8]) separates the signing key from the account logic, enabling agents to sponsor gas, batch operations, and authorize delegated execution. The agent-to-agent finance literature [1] surveys blockchain payment and trust infrastructure for autonomous agents. Our work assumes such an account layer as the substrate and focuses on the higher-level question of how two accounts that have never met reach trust: guarantee, adjudicate, and build reputation.

2.4 On-chain reputation and attestation

The Ethereum Attestation Service (EAS) [9] provides a general on-chain attestation registry. AgentReputation [2] proposes a decentralized reputation framework for agentic AI. Recent datasets of blockchain-registered AI agents [10] indicate early adoption of agent identity standards. These works establish that reputation *can* be recorded on-chain, but leave open who is authorized to write, how truthful attestations are incentivized, and how reputation is priced into risk. Our reputation hub (Section 5.2) closes the write-incentive gap by restricting writers to audited escrow/voting contracts (no self-rating) and by coupling scores to guarantee pricing.

2.5 Decentralized arbitration and guarantees

Decentralized dispute resolution has been explored by Kleros [11] (a jury selected from a set of paid, random jurors), Aragon Court [12] (aragon.org/dish), and UMA's optimistic oracle [13] (price oracle with a challenge period). Kleros and Aragon Court select jurors by *lottery* and reward coherent majorities; UMA incentivizes truthful price reporting through disputes and forfeits. These mechanisms share the Schelling-point insight — that truthful majorities are self-enforcing — but

none of them *closes the loop* with reputation and guarantee pricing: a Kleros verdict is final and does not update a merchant's future cost of doing business. On the insurance side, Nexus Mutual [14] runs a mutual cover pool with a claims-assessment community. Our contribution is to make the guarantee itself (escrow plus guarantor stake) the stake that voters price, and to make every verdict feed a reputation score that prices the next guarantee — a three-way closed loop (Section 4.4).

2.6 Trust research specific to agents

Recent work on trustworthy LLM agents [15, 16, 17] and on binding agent identities [18] converge on the need for attestation, insurance, and verifiable accountability. ERC-8004 (trustless agents) [19] sketches an interoperability standard. We position this paper as the first to combine, end-to-end and with a formal incentive analysis, (i) a registered identity, (ii) a guaranteed escrow, (iii) staked-voting arbitration, and (iv) reputation-driven guarantee pricing in a single protocol whose contracts and empirical validation are public.

3 The ITA Architecture

This section describes *what* the protocol records and *how* it holds agents accountable. The incentive layer that explains *why* rational agents behave honestly is the subject of Section 4; the concrete contract implementation is Section 5.

3.1 Design goals and threat model

We model three classes of principals: agents A_i (autonomous trading entities), certification and guarantee providers S (guarantors, premium setters), and adjudication/enforcement entities E (voters, settlement executors). The system is designed to uphold six trust properties: *identity authenticity* (every agent maps to a real principal), *non-repudiation* (commitments are provably attributable), *traceability* (outcomes are auditable on-chain), *fairness* (disputes are decided by a verifiable, non-capturable process), *incentive compatibility* (honest behavior is a dominant/equilibrium strategy), and *privacy adjustability* (only necessary claims are exposed). It defends against four canonical attacks: Sybil attacks (mass identity fabrication), reputation manipulation (self-rating or score farming), delinquency-and-exit (misbehaving under one identity then abandoning it), and single-point-authority abuse (a centralized credit adjudicator who simultaneously scores, revokes, and investigates).

3.2 Layer 1 — Identity

The identity layer gives every agent a verifiable, principal-bound identity. Rather than introducing new cryptographic primitives, it composes mature standards — W3C DIDs/VCs, soulbound tokens (EIP-5192/4973), account abstraction (ERC-4337), and proof-of-personhood — into a single on-chain registry. In our implementation, AgentRegistry mints an ERC-721 Agent ID per registered agent, binds it to the caller (the responsible principal), and charges a *registration fee* to price identity creation and deter Sybils. The registry thereby answers Q1's identity side: agents must "obtain a license before trading."

3.3 Layer 2 — Trust

The trust layer records, qualifies, and prices evidence of good behavior. It stores reputation profiles as on-chain attestations (the role of EAS in the standards survey), supports contextualized multi-

dimensional scoring, and authorizes delegated claims via VCs. In our implementation, ReputationHub maintains per-agent multi-dimensional profiles (completions, defaults, dispute wins/losses), restricts writers to audited escrow/voting contracts (no self-rating), and exposes a scalar score that prices future guarantees (Section 4.4).

3.4 Layer 3 — Accountability

The accountability layer guarantees funds, adjudicates disputes, and sanctions misbehavior. It combines guaranteed escrow (the role of Nexus Mutual-style insurance), decentralized arbitration (the role of Kleros/UMA), and forfeiture/revocation. In our implementation, GuaranteeEscrow escrows trade funds and holds guarantor stakes that are forfeited on default, and SchellingVoting adjudicates disputes by staked community vote whose verdict drives escrow release and reputation update.

3.5 From centralized adjudicator to decentralized trust infrastructure

A central design thesis, echoing the survey's analysis, is that a *social-credit-style* centralized adjudicator — a single authority that scores, revokes, and investigates — is a single point of abuse, monopoly, privacy, and fairness failure. The ITA architecture therefore shifts the design center from "centralized credit adjudicator" to "operator and rule-setter of decentralized trust infrastructure": trust is maintained by verifiable code and multi-party governance rather than by the subjective endorsement of a single authority. This motivates the decision that dispute adjudication (the arbiter of what counts as good behavior) be decentralized staked voting rather than a privileged contract owner — the subject of Section 4.

4 Mechanism Design

This chapter formalizes the incentive layer of AgentTrust. Whereas the identity–trust–accountability (ITA) architecture of the preceding chapter describes *what* the protocol records and *how* it holds agents accountable, here we make precise *why* rational agents choose to behave honestly. The core object of study is the *Schelling-point reputation community*: a set of staked voters who adjudicate trade disputes, whose verdicts drive both the release of escrowed funds and the updating of on-chain reputation, and whose reputation scores in turn price the very guarantees that make future trades safe. The mechanism is instantiated by four production Solidity contracts – AgentRegistry, GuaranteeEscrow, SchellingVoting, and ReputationHub – and every parameter cited below matches the deployed bytecode exactly.

We proceed as follows. Section 4.1 states the design goals and the adversarial model, and fixes the participants, strategy space, and utility functions used throughout. Section 4.2 gives the formal definition of the mechanism. Section 4.3 presents the game-theoretic analysis: honest voting as a best response (Lemma 4.7), incentive compatibility (Theorem 4.8), collusion resistance (Theorem 4.10), Sybil resistance (Theorem 4.12), and deterministic fund safety (Lemma 4.15), together with a quantitative justification of the two-thirds supermajority rule. Section 4.4 closes the reputation–guarantee pricing loop, and Section 4.5 contrasts the design with existing decentralized adjudication and attestation systems.

4.1 Design Goals and Threat Model

Participants.

Let $\mathcal{I} = \{1, \ldots, n\}$ be the set of registered *agents*. Each agent $i \in \mathcal{I}$ holds a non-fungible identity issued by AgentRegistry; the registry binds i to a unique on-chain principal $o(i) \in \mathbb{A}$ (the entity legally responsible for i's actions), so every act by an agent is attributable to a real-world principal [19]. Three roles interact within the mechanism:

1. *Traders* – a buyer agent b and a seller agent s who enter into a guaranteed commerce transaction of value x (in wei).
2. *Guarantors* – principals who post capital to insure the buyer against seller default, in exchange for a premium.
3. *Voters* – community members who stake capital and vote on disputed outcomes; their collective verdict settles the case.

The trader and guarantor roles are exercised through GuaranteeEscrow; the voter role through SchellingVoting. All write access to reputation is mediated by ReputationHub, which maintains an access-control list of authorized callers (the escrow and voting contracts), thereby precluding self-attestation by construction.

Design goals.

We seek a mechanism satisfying five properties.

G1 (Decentralized adjudication). No single authority determines the winner of a dispute; outcomes are decided by a staked community supermajority. The platform operator retains no privileged arbitration role beyond bootstrapping (opening cases in the MVP).

G2 (Incentive compatibility). Truthful voting is a (strict) Bayesian–Nash equilibrium: given that the community converges on the true state, no voter can profit by misreporting her private information.

G3 (Sybil resistance). Neither identities nor votes can be multiplied for free: identity creation is gated by a registration stake, and every vote requires capital lock-up.

G4 (Collusion resistance). A coordinated minority cannot overturn the honest majority's verdict; forcing an outcome requires a strict two-thirds supermajority of capital-weighted votes.

G5 (Deterministic fund safety). At every state of the lifecycle, funds are conserved: the sum of deposits equals the sum of distributable value up to a bounded integer-division dust term, and no state transition can lock or leak value.

Threat model.

We assume a fully rational adversary who controls a bounded fraction of registered agents and/or of staked voting capital, and who may deviate arbitrarily (computationally unbounded in strategy, economically budgeted in capital). The protocol must remain secure against the following attack classes, which specialize the abstract ITA threat catalogue of the preceding chapter.

T1 (Sybil). An adversary creates many fake agent identities and/or voting addresses to dominate adjudication or to inflate reputation. This is the classic Sybil attack [20]; we bound it with registration stakes, per-vote capital lock-up, and a minimum quorum.

T2 (Reputation manipulation). An adversary rates herself (self-rating), colludes to rate conspirators, or otherwise inflates a reputation profile. We exclude self-rating by ACL-gated writes, and make ratings costly by routing all writes through the escrow/voting contracts where the write is driven by a staked verdict or an escrowed outcome.

T3 (Delinquency and exit). An agent earns a series of high-value trades, defaults once, and abandons the identity to relaunch under a fresh one (*reputation laundering*). We bound this by the non-recoverable registration stake, by the permanence of the on-chain record, and by reputation-gated guarantee pricing that prices a fresh identity at the uninsurable default rate (Section 4.4).

T4 (Single-point authority abuse). Any participant who holds exclusive arbitration or write authority can extract rent or mis-route funds. The design minimizes such points: in the full protocol, the arbitration authority is exercised by SchellingVoting (community supermajority), not by a privileged key.

Strategy space and utility.

Fix a disputed case C. Each voter i chooses an action $a_i \in \mathcal{A} = \{B, S, \perp\}$, where B (BUYER), S (SELLER), and $\perp$ (ABSTAIN) denote the sides of the dispute she supports; choosing a_i requires depositing a stake $v > 0$ wei (SchellingVoting.vote enforces `msg.value` $= v$ exactly). Voter utility is quasi-linear, $u_i(a) = P_i(a) - v$, where P_i is the settlement payment defined in Section 4.2. We model information with a two-point state space $\theta \in \{B, S\}$ (the true party at fault); each voter receives a private signal $\sigma_i \in \{B, S\}$ with accuracy $q = \Pr[\sigma_i = \theta \mid \theta] > 1/2$, conditionally independent across voters. This is the standard Condorcet jury information structure [21], which we instantiate under a staked-payment payoff in place of the classical "correct decision" utility.

4.2 Formal Mechanism Definition

We now define the mechanism formally. Throughout, monetary quantities are denominated in wei; "$\lfloor \cdot \rfloor$" is integer floor; all divisions are integer divisions exactly as performed by the EVM.

Definition 4.1 (Dispute case)**.** *A* dispute case *is a tuple*

$$C \;=\; (\tau,\; b,\; s,\; v,\; t_d,\; m_B,\; m_S,\; m_\perp,\; w,\; e),$$

where τ is the trade identifier (`tradeId`), b, s are the buyer and seller agent identifiers, $v > 0$ is the per-vote stake (`stake`), t_d is the voting deadline (a block timestamp), $(m_B, m_S, m_\perp)$ are the numbers of votes cast for BUYER, SELLER, and ABSTAIN respectively, $w \in \{B, S, \perp\}$ is the winner (with $w = \perp$ denoting a void case), and $e \in \{0, 1\}$ is the effectiveness flag. The protocol constrains a case by: (i) one vote per address (`hasVoted` is a per-case flag); (ii) votes accepted only during the open window $(\cdot,\; t_d)$; and (iii) at most one case per trade (`tradeHasCase` prevents duplicate litigation), opened only while the underlying trade is in the `DISPUTED` escrow state.

Definition 4.2 (Voting game)**.** *Given a case C, the induced* voting game *is*

$$\Gamma_C = \big(\mathcal{N},\; \mathcal{A},\; S,\; P,\; u\big),$$

where $\mathcal{N}$ is the set of eligible voters, $\mathcal{A} = \{B, S, \perp\}$ is the pure-action space, S is the deposit schedule requiring exactly v wei per vote, P is the settlement payment function of Definition 4.4, and $u_i(a) = P_i(a) - v$ is the quasi-linear utility of voter i. A mixed strategy of voter i is a distribution over $\mathcal{A}$; the Bayesian game extends Γ_C with the type space of signals (σ_i) and the common prior over $(\theta, \sigma_1, \ldots, \sigma_{|\mathcal{N}|})$.

Definition 4.3 (Adjudication rule)**.** *Let $T = m_B + m_S$ be the number of* effective *votes (abstentions are excluded). The adjudication rule R maps the vote profile to (w, e) as follows, with $\kappa =$ `MAJORITY_BPS`$/10000 = 0.65$ and $\nu =$ `MIN_VOTERS`$= 3$:*

1. *If $T = 0$, set $w = \perp$ and $e = 0$ (no votes cast; the contract short-circuits the majority test, which would otherwise vacuously hold because $0 \geq 0$).*
2. *Otherwise, for each side $X \in \{B, S\}$ define the supermajority predicate*

$$\mathsf{maj}(X) \iff m_X \cdot 10000 \;\geq\; T \cdot \texttt{MAJORITY\_BPS},$$

   and set $w = B$ if $\mathsf{maj}(B)$, else $w = S$ if $\mathsf{maj}(S)$, else $w = \perp$.
3. *Finally, $e = 1 \iff \big(T \geq \nu \;\wedge\; w \neq \perp\big)$; otherwise $e = 0$ and the case is* void*.*

Because $\kappa = 0.65 > 1/2$, the two predicates $\mathsf{maj}(B)$ and $\mathsf{maj}(S)$ cannot both hold for any $T \geq 1$; the rule is therefore well-defined. Note that $\kappa = 0.65$ is an integer-safe engineering approximation of the exact two-thirds threshold $\frac{2}{3} \approx 0.6667$ (Section 4.3); the contract's comment documents this choice as avoiding boundary integer-division ambiguity.

Definition 4.4 (Settlement payments)**.** *Let w be the winner and e the effectiveness of a settled case. Let $m_w = m_B$ if $w = B$, else $m_w = m_S$; let $m_\ell = T - m_w$. The settlement payment function P is:*

1. ***Effective case*** *($e = 1$). Each majority voter receives*

$$P_i = v + \left\lfloor \frac{v \cdot m_\ell}{m_w} \right\rfloor,$$

   i.e., her principal plus a pro-rata share of the forfeited minority pool (`claimReward`, $T \geq \nu$ guaranteed). Each minority voter receives $P_i = 0$ (her stake is forfeited). Each ABSTAIN voter receives $P_i = v$ (`claimRefund`).
2. ***Void case*** *($e = 0$). Every voter – majority, minority, or abstainer – receives $P_i = v$; no rewards are distributed.*

The integer-division remainder $\delta = (v \cdot m_\ell) \bmod m_w$ remains in the contract; it is bounded by $\delta < m_w$ wei and is a known, documented limitation of the MVP (a batched-settlement refinement in the full design removes it).

Definition 4.5 (Escrow linkage)**.** *On settlement, the voting contract invokes the escrow's dispute-resolution entry point,*

$$\texttt{escrow.resolveDispute}(\tau,\ \texttt{verdict},\ \texttt{share}),$$

on the underlying trade τ, which releases escrowed value and updates reputation as follows. The escrow enforces the state machine CREATED $\to$ FUNDED $\to$ GUARANTEED $\to$ DELIVERED $\to$ DISPUTED $\to$ RELEASED/RESOLVED, with four one-day windows (funding, guarantee, delivery, and confirmation); a case can be opened only while the trade is in the `DISPUTED` state (Definition 4.1).

1. *If $w = B$ (including the conservative default of a void case), the verdict is* `BUYER_WINS` *with* `share` $= 10000$. *The buyer receives* `buyerRefund` $+$ `stake`, *where* `buyerRefund` $= \lfloor x \cdot \texttt{share}/10000 \rfloor$ *and* `stake` $= \lfloor x \cdot \gamma/10^{18} \rfloor$ *is the guarantor's posted collateral; the seller receives the residual $x -$* `buyerRefund` *if positive. The seller is recorded as* `LOST`.

2. *If $w = S$, the verdict is* `SELLER_WINS` *with* `share` $= 0$. *The seller receives $x - \pi$ (amount less the premium) and the guarantor receives* `stake` $+ \pi$; *the seller is recorded as* `WON`.

The verdict thereby releases *escrowed value, and simultaneously feeds the reputation record (Definition 4.6); neither can occur without the other. All amounts are verified to sum to the escrow's holding $x +$* `stake` *in every branch (Lemma 4.15).*

Definition 4.6 (Reputation update rule)**.** *ReputationHub maintains, per agent, a four-dimensional behavior profile $\rho = (c, d, w, l)$ counting* `COMPLETED`, `DEFAULTED`, `WON`, *and* `LOST` *outcomes, written exclusively by authorized callers (the escrow and voting contracts). From the seller's perspective: a trade confirmed or auto-released records* `COMPLETED`; *a seller default on the delivery window records* `DEFAULTED`; *a dispute won or lost records* `WON` *or* `LOST`. *The scalar score is*

$$\mathsf{score}(\rho) \;=\; 100 \;-\; 100 \cdot \frac{d}{\theta} \;-\; 50 \cdot \frac{l}{\theta}, \qquad \theta = c + d + w + l,$$

clipped to $[0, 100]$, and score $= 50$ *for a new agent with $\theta = 0$. A default reduces the score by up to 100; a lost dispute by up to 50; completed trades and won disputes never reduce it.*

4.3 Game-Theoretic Analysis

We analyze the mechanism in three parts: the per-voter incentive to tell the truth, the system-level barriers to coalition and Sybil attacks, and the fund-safety invariant. Throughout, we fix a case C with stake v, and let h (resp. c) denote the number of votes cast by honest (resp. adversarial) voters.

Signal structure. Each honest voter i receives signal σ_i with $\Pr[\sigma_i = \theta \mid \theta] = q > 1/2$, conditionally independent across voters. Honest voters are assumed to be rational and to coordinate on the *truthful* Schelling point: they vote the side they believe is at fault, because the majority side collects the minority's forfeited stakes [22, 23]. We first show that this behavior is indeed a best response to honest others.

Lemma 4.7 (Honest voting is a best response)**.** *Fix a voter i holding signal σ_i with accuracy $q > 1/2$, and suppose every other voter votes her own signal. Then voting σ_i strictly maximizes i's expected settlement payoff among $\{B, S\}$, and strictly dominates abstention whenever the other voters carry the correct side to a supermajority. Concretely,*

$$\mathbb{E}[u_i \mid a_i = \sigma_i] \;-\; \mathbb{E}[u_i \mid a_i = \bar{\sigma}_i] \;=\; v\left(1 + \frac{m_\ell}{m_w}\right)\left(p_{\sigma_i} - p_{\bar{\sigma}_i}\right) \;>\; 0,$$

where p_X is the probability that voter i is in the majority when voting X, and $p_{\sigma_i} - p_{\bar{\sigma}_i} \geq (2q-1) - \varepsilon_n$, with $\varepsilon_n \to 0$ as the number of voters grows.

Proof. The deposit v is sunk at the time of the vote, so i maximizes her expected settlement payoff. Let m_B, m_S denote the counts of the *other* voters' votes, $m_B + m_S = n - 1$. If i votes B, the effective totals become $(m_B + 1, m_S)$; by rule R she is in the majority iff $m_B + 1 \geq \kappa\,(m_B + m_S + 1)$. If she

votes S, she is in the majority iff $m_S + 1 \geq \kappa\,(m_B + m_S + 1)$. Except on the one-vote-wide pivotal region where these two inequalities differ, the winner is independent of i's vote and $p_B = p_S$; the entire payoff gap is concentrated on the pivot event.

Conditional on the true state, the other honest votes are conditionally i.i.d. with success probability q, hence $m_B \mid \theta = B \sim \mathsf{Bin}(n-1, q)$ first-order stochastically dominates $m_B \mid \theta = S \sim \mathsf{Bin}(n-1, 1-q)$. Because i's own signal satisfies $\Pr[\theta = \sigma_i] = q > 1/2$, she weights the correct-state branch more heavily, giving $p_B > p_S$ when $\sigma_i = B$ (and symmetrically). Concretely, the probability that her vote is pivotal and decisive for the truthful side exceeds the analogous probability for the untruthful side by at least $2q - 1$, up to the vanishing slack ε_n from the law of large numbers. By Definition 4.4, her expected net payoff under vote X is $v\big(p_X(1 + m_\ell/m_w) - 1\big)$, so the gap above equals $v(1 + m_\ell/m_w)(p_B - p_S) > 0$. For abstention: if the other voters already carry the correct side to a supermajority, voting one's signal yields a strictly positive expected net payoff (the pivot bonus), whereas abstention yields exactly 0; abstention is therefore strictly dominated. □

Theorem 4.8 (Incentive compatibility: truthful voting is an equilibrium)**.** *Let every voter's signal accuracy satisfy $q > 1/2$, and suppose the honest community commands at least a fraction $\kappa = 0.65$ of the effective votes in expectation. Then the profile in which every voter votes her signal is a strict Bayesian–Nash equilibrium: no voter can strictly improve by any unilateral deviation. Moreover, the resulting verdict coincides with the true state with probability converging to* 1 *as the number of voters grows, and the truthful profile is the unique* signal-sustained *Schelling point among the coordination equilibria of Γ_C.*

Proof. **(Equilibrium.)** Take any voter i. By Lemma 4.7, given that all other voters vote their signals, every $a_i \in \{B, S, \perp\}$ with $a_i \neq \sigma_i$ yields strictly lower expected utility than $a_i = \sigma_i$ (abstention being dominated whenever the honest supermajority materializes, which occurs with probability $1 - \varepsilon$ by the assumption that the honest block commands a $\geq \kappa$ fraction). Hence no unilateral deviation is profitable and the inequality is strict, so the truthful profile is a strict Bayesian–Nash equilibrium.

(Correctness.) Conditional on $\theta = B$, the honest votes are i.i.d. with success probability $q > 1/2$, so by the law of large numbers the fraction of B votes among honest voters tends to q. Since the honest block commands at least $\kappa = 0.65$ of all effective votes and $q > \kappa$ is not required for this step – the block simply needs its own supermajority to exceed κ – the B share of total effective votes exceeds κ with probability tending to 1; hence R returns $w = B$ with probability tending to 1. The argument for $\theta = S$ is symmetric.

(Schelling point.) Other coordination profiles (e.g., all voters choosing B irrespective of signals) are Nash equilibria as well, since no single voter can change the outcome if the rest coordinate. But any such coordination equilibrium is uncorrelated with the true state: it rewards the majority regardless of the merits. The truthful profile is the unique equilibrium in which (i) each voter's action is a best response to a signal, (ii) the outcome is statistically correct, and (iii) expected payoffs are strictly higher conditional on any realization where the coordinated and truthful winners differ (the truthful majority collects a forfeiture bonus, whereas the mis-coordinated majority gains nothing real – the escrow still releases value to the wrong party only for a single case). Following Schelling's focal-point theory, self-interested voters with a common prior converge on the signal-sustained profile because it is the unique one justifiable to the community [22, 24, 23]. □

Remark 4.9 (Scope of the dominance claim)**.** *Lemma 4.7 establishes honesty as a strict best response to honest others, not as a dominant strategy against a fully adversarial electorate (a lone honest voter against $n - 1$ adversarial votes cannot force the truth). This is the standard truthful-*

equilibrium property of Schelling-style staked voting [24, 23]; the system-level *protection against adversarial electorates is exactly the quantitative barrier of Theorem 4.10 and Theorem 4.12.*

Theorem 4.10 (Collusion resistance)**.** *Let h honest and c adversarial voters participate in a case, with $n_e = h + c$ effective votes. To force a verdict contrary to the will of the h honest voters, the coalition must command*

$$c \;\geq\; \left\lceil \frac{13}{7}\, h \right\rceil \;\approx\; 1.857\, h \qquad (\textit{i.e., } c \geq 2h \textit{ under the ideal threshold } \tfrac{2}{3}),$$

effective votes, i.e., a strict supermajority of at least $0.65\, n_e$. Consequently: (i) any coalition of at most 35% of the effective voters is strategically harmless – it can neither determine nor veto an outcome, and its stakes are forfeited; and (ii) the minimum capital an attacker must lock is $c \cdot v \geq \lceil 13h/7 \rceil \cdot v$, and a rational attacker will not mount an attack when this exceeds the recoverable dispute value ($c \cdot v > x + \pi$).

Proof. For the coalition's preferred side X to win, rule R (Definition 4.3) requires $m_X \geq \kappa T$. In the best case for the coalition, every honest voter opposes it, so $m_X = c$ and $T = c + h$; the binding inequality is $c \geq 0.65\,(c + h)$, which rearranges to $c \geq \frac{13}{7} h$. If the coalition fails this bound, the honest side either carries the case (when $h \geq \frac{13}{7} c$) or the case is void (when neither side reaches 0.65); in the latter event the conservative default releases the escrow to the buyer (Section 4.3), never to the coalition's preferred verdict against an honest block. Hence the coalition's *only* path to a dictated verdict requires the supermajority of $0.65\, n_e$, i.e., $c \geq \lceil 13h/7 \rceil$. Claim (i) follows because any $c \leq 0.35\, n_e$ fails the bound; such votes join a losing minority and are forfeited by Definition 4.4. Claim (ii) follows from the capital lock: the attacker risks $c \cdot v$ wei (plus the registration cost of Theorem 4.12), and the total value extractable from a captured verdict is at most the trade value plus premium, $x + \pi$. When $c \cdot v > x + \pi$, expected attack profit is strictly negative for any win probability bounded away from 1 – and by the barrier above it is not 1 unless c meets the supermajority, at which point the cost dominates. □

Remark 4.11 (Void-case vetoing)**.** *A coalition that cannot reach* 0.65 *can still* veto *an honest verdict by preventing the honest side from reaching κ; but this requires diluting the effective total with abstentions or adversarial votes to force a void. Since void cases default to the buyer and refund every stake, the veto yields the coalition at most the conservative buyer-wins outcome, never a seller-favorable verdict against an honest block. Moreover, inducing honest voters to abstain requires compensating them for the positive expected bonus of Lemma 4.7, adding a bribe cost to the attack. The veto therefore cannot subvert the buyer-insurance guarantee that motivates the escrow.*

Theorem 4.12 (Sybil resistance)**.** *Let an adversary create k Sybil identities. Each identity costs* `registrationFee` *wei to register in* AgentRegistry *and is permanently bound to a real principal; each effective vote additionally requires a capital lock of v wei in* SchellingVoting*; and no case can be decided by fewer than* `MIN_VOTERS` $= 3$ *effective votes. By Theorem 4.10, capturing a case against $h \geq 1$ honest voters requires*

$$k \;\geq\; \left\lceil \frac{13}{7}\, h \right\rceil,$$

so the minimum attack expenditure is at least $\lceil 13h/7 \rceil \cdot (v + $* `registrationFee` *$)$ wei. Because identity creation and voting both demand real capital that is non-reusable while locked, adversarial voting power scales linearly *in expenditure, and the quorum $\nu = 3$ rules out one- or two-identity Sybils deciding a case.*

Proof. Three protocol constraints jointly bound Sybil power. First, AgentRegistry.registerAgent requires a payment of at least `registrationFee` and binds the new identity to the caller, so each Sybil identity is a paid, attributable liability rather than a free vote. Second, SchellingVoting.vote requires the exact per-vote deposit `msg.value` $= v$ and permits one vote per address, so each effective Sybil vote locks a full unit of capital that cannot be reused until the case settles. Third, rule R requires $T \geq \nu = 3$ effective votes for a case to be effective, so a Sybil of size one or two cannot manufacture a verdict at all. Combining these with Theorem 4.10, which mandates $c \geq \lceil 13h/7 \rceil$ to capture a case, yields the stated expenditure bound. Linearity follows because neither the identity cost nor the vote deposit can be amortized across identities while locked. □

Remark 4.13 (MVP vs. full design)**.** *The MVP's SchellingVoting enforces the* capital *Sybil barrier (exact per-vote stake, one vote per address, $\nu = 3$); the identity barrier (the requirement that voters hold registered agent IDs) is enforced by the escrow/trading path and is a stated refinement of the full protocol, which binds every vote to a registered, fee-paid identity. Both barriers are needed for the full bound; the capital barrier alone already prevents cost-free vote multiplication.*

Remark 4.14 (Sufficiency, not exactness, of the Sybil bound)**.** *The bound $k \geq \lceil 13h/7 \rceil$ of Theorem 4.12 is a* sufficient *condition: it guarantees the coalition captures the case (its preferred side reaches κ with certainty). It is not the smallest coalition that yields a* high-probability *capture, because honest voters err with probability $1 - q$ (Definition 4.1), leaking a share of honest votes to the adversary's side. Our Monte-Carlo evaluation (Section 5.4) confirms this: at $h = 14$ honest voters and $q = 0.85$, the flip probability reaches 0.65 already at $k = 22$, below $\lceil 13h/7 \rceil = 26$, and certainty only at $k = 26$. The bound therefore overstates the necessary adversarial expenditure in noisy electorates; the exact capture set is the threshold boundary $m_X \geq \kappa T$ of Definition 4.3, which the protocol enforces regardless of coalition size. For the security claim this direction is the safe one: the mechanism never needs* more *adversarial capital to resist an attack than the bound indicates.*

Lemma 4.15 (Deterministic fund safety)**.** *For any case C, the total staked pool is $(m_B + m_S + m_\perp) \cdot v$ wei. The settlement payments of Definition 4.4 satisfy*

$$\sum_{i \in \mathcal{N}} P_i \;=\; \begin{cases} (m_B + m_S + m_\perp) \cdot v - \delta, & e = 1,\ \delta = (v\, m_\ell) \bmod m_w, \\ (m_B + m_S + m_\perp) \cdot v, & e = 0, \end{cases}$$

with $0 \leq \delta < m_w$. In every effective settlement the majority reclaims its principal and divides the minority's forfeited stakes; in every void settlement all stakes are refunded. Hence the voting contract never over-disburses and never accumulates unaccounted value beyond the bounded dust term δ. At the escrow layer, the same invariant holds: the escrow holds $x +$ `stake` wei and disburses exactly $x +$ `stake` across every outcome (Table 1).

Proof. In an effective case ($e = 1$), m_w majority voters each receive $v + \lfloor v m_\ell / m_w \rfloor$, m_ℓ minority voters receive 0, and $m_\perp$ abstainers receive v. The total is

$$m_w \left(v + \left\lfloor \tfrac{v m_\ell}{m_w} \right\rfloor \right) + m_\perp v \;=\; m_w v + (v m_\ell - \delta) + m_\perp v \;=\; (m_B + m_S + m_\perp) v - \delta,$$

since $m_w + m_\ell = m_B + m_S$. In a void case ($e = 0$), every voter claims v, summing to the full pool with $\delta = 0$. The dust bound $\delta = (v m_\ell) \bmod m_w < m_w$ follows from the integer remainder. For the escrow: enumerating the outcome classes (success, seller wins, buyer wins in full/partial, and the two refund paths) shows each distributes $x +$ `stake` or, in the pre-guarantee refund, the full x held at that state; the premium π is never deposited and is netted from the seller's share, so it cannot be lost. □

Table 1: Conservation of the escrow's holding $x + \texttt{stake}$ (stake $= \lfloor x\gamma/10^{18} \rfloor$, premium π, buyer share $\beta/10000$). Across every outcome the sum of payouts equals the holding exactly.

Outcome	Payout to seller	Payout to guarantor / buyer
Success (confirm / auto-release)	$x - \pi$	guarantor: $\texttt{stake} + \pi$
Seller wins dispute	$x - \pi$	guarantor: $\texttt{stake} + \pi$
Buyer wins dispute (full, $\beta = 10000$)	0	buyer: $x + \texttt{stake}$
Buyer wins dispute (partial, $\beta < 10000$)	$x - \beta x/10000$	buyer: $\beta x/10000 + \texttt{stake}$
Refund before guarantee	—	buyer: x
Seller default (delivery timeout)	0	buyer: $x + \texttt{stake}$

The conservative default on void cases. When a case is void – no votes, fewer than ν effective votes, or no side reaching the supermajority – rule R returns $w = \perp$ and the escrow *conservatively defaults to the buyer*: verdict `BUYER_WINS`, `share` $= 10000$ (Definition 4.5). The buyer is refunded her principal plus the guarantor's forfeited stake; the seller's claim to payment is denied. This asymmetry is economically deliberate and not an artifact:

1. **Insurance semantics.** The guarantor has voluntarily accepted the role of insuring the buyer against seller default. When the community cannot establish the seller's claim to a supermajority, the seller's delivery claim is treated as unsubstantiated, and the insurance settlement is exercised in the buyer's favor – the standard burden-of-proof allocation in which the party seeking payment must establish entitlement.

2. **No value lock.** Funds are released deterministically rather than frozen; G5 holds unconditionally, including on void paths.

3. **Guarantor discipline.** The default-buyer outcome makes a guarantor who underpriced a risky seller absorb the loss, aligning guarantee pricing (Section 4.4) with the community's future verdicts and deterring adverse selection into the guarantee market.

Two caveats are documented. First, a void case still records a `LOST` outcome for the seller through the default path, which is a deliberate MVP simplification; the full protocol distinguishes *void* from *adjudicated loss* so that a seller who merely faced an abandoned dispute is not penalized. Second, the default gives a buyer who files a frivolous dispute a "win by default" if the community fails to respond; this is bounded because the honest community needs only $\nu = 3$ votes and a supermajority to rule `SELLER_WINS` (Lemma 4.7 makes truthful response a best response), and because repeated frivolous disputes are themselves on-chain-visible and subject to the community's reputational scrutiny.

Why a two-thirds supermajority? The threshold $\kappa = \texttt{MAJORITY\_BPS}/10000 = 0.65$ is chosen for three quantitative reasons, none of which hold at the bare-majority rule $\kappa = 1/2$.

(a) Resisting a one-third malicious minority. For any contested case, the honest block wins iff its vote share exceeds κ. With $\kappa = 1/2$, an adversary holding just over half of the votes wins every case it contests, and the honest block must exceed half to survive. With $\kappa = 0.65$, an adversary controlling up to 35% of the effective votes can *never* determine an outcome (Theorem 4.10); the mechanism tolerates a coordinated malicious minority of roughly one third while still producing the correct verdict whenever honest voters command a ≥ 0.65 share.

(b) Capping the required honest supermajority. A threshold close to 1 (unanimity) would render the mechanism fragile: a small honest abstention or a little signal noise would void legitimate cases.

At $\kappa = 0.65$, the honest block needs only two thirds of the votes, so up to one third of the electorate may abstain, err, or collude without voiding a valid verdict – keeping the truthful equilibrium of Theorem 4.8 sustainable without near-unanimity.

(c) Optimizing the voting incentive at the boundary. The per-winner bonus is $v \cdot m_\ell / m_w$, which at the decision boundary equals $v \cdot (1-\kappa)/\kappa$: a 50% return at $\kappa = 2/3$, 100% at $\kappa = 1/2$, and 33% at $\kappa = 3/4$. The supermajority rule thus preserves a strong participation bonus at the exact margin where a verdict is contested, while the bare-majority rule's larger bonus buys no collusion resistance at all.

The engineering constant `MAJORITY_BPS` $= 6500$ is the integer-safe approximation of $\frac{2}{3} \approx 6667/10000$; it lowers the adversarial requirement marginally (factor $\frac{13}{7} \approx 1.857$ instead of 2) at negligible security cost, in exchange for exact integer arithmetic and a slightly smaller void region near the threshold boundary.

The choice is borne out quantitatively by the Monte-Carlo evaluation of Section 5.4: scanning the threshold with $n = 15$ honest voters and $q = 0.7$ yields a correct-verdict rate of 0.947 at $\kappa = 0.5$, 0.721 at $\kappa = 0.65$, and 0.512 at $\kappa = 0.7$, while the void rate climbs from 0 to 0.275 to 0.487. The supermajority rule thus trades at most 0.23 of correctness for immunity to a one-third malicious minority, and raising κ beyond 0.65 buys no further safety while voiding nearly half of all cases. The same evaluation confirms the 35% tolerance of reason (a) directly: with $n = 21$ and $q = 0.85$, a coalition controlling 38% of the effective votes flips the verdict with probability below 1%, and certainty is reached only at the 0.65 supermajority boundary.

4.4 The Reputation–Guarantee Pricing Loop

The design's central novelty is not adjudication per se but the *closed loop* that connects verdicts to future commerce: each settlement updates a reputation profile, reputation prices the guarantee that makes the next trade safe, and the guarantee determines whether low-reputation agents can transact at all.

Definition 4.16 (Guarantee contract and pricing)**.** *A guarantee is a pair (γ, π) with coverage $\gamma \in (0, 2]$ (fraction of x, up to 200%) and premium $\pi \geq 0$ wei, subject to $\pi \leq x$ (`premium` $\leq$ `amount`, enforced to prevent underflow of the seller's share $x - \pi$). The guarantor deposits `stake` $= \lfloor x\gamma / 10^{18} \rfloor$ wei into* GuaranteeEscrow*. Let $p_D(\rho)$ be the probability that a seller with profile ρ defaults. A rational guarantor participates only if its expected net payoff is non-negative:*

$$(1 - p_D)\,\pi \;-\; p_D\,\texttt{stake} \;\geq\; 0 \iff \pi \;\geq\; \frac{p_D}{1 - p_D} \cdot \texttt{stake}.$$

We call $\pi^*(\rho, \gamma) = \dfrac{p_D}{1 - p_D} \cdot$ `stake` the competitive premium floor*.*

To instantiate the floor we need an estimator of the default probability from the on-chain profile. The natural choice is the empirical default rate $\hat{p}_D(\rho) = d/\theta$, which is exactly the quantity the reputation score is built on. Substituting and using the identity $d/\theta = (100 - \mathsf{score}(\rho))/100 - l/(2\theta)$, a first-order, monotone estimator is $\hat{p}_D(s) = (100 - s)/100$ for a score $s = \mathsf{score}(\rho) \in [0, 100]$; it is a valid probability, equals 0 at the maximal score, and equals 1 at the minimal score.

Proposition 4.17 (Monotone guarantee pricing)**.** *Fix a trade value x and coverage $\gamma > 0$, and set $\hat{p}_D(s) = (100 - s)/100$. The competitive premium floor*

$$\pi^*(s) \;=\; \frac{\hat{p}_D(s)}{1 - \hat{p}_D(s)} \cdot \texttt{stake} \;=\; \texttt{stake} \cdot \frac{100 - s}{s}$$

is strictly decreasing in the reputation score s on $(0, 100]$: $\dfrac{\partial \pi^*}{\partial s} = -100\,\texttt{stake}/s^2 < 0$. *Consequently the seller's net proceeds $x - \pi^*(s)$ are strictly increasing in s, and a guarantor who quotes $\pi < \pi^*(s)$ is strictly loss-making in expectation, while any $\pi \geq \pi^*(s)$ yields non-negative expected payoff.*

Proof. The estimator $\hat{p}_D(s)$ is strictly decreasing in s, and the floor $\pi^* = \texttt{stake} \cdot \hat{p}_D/(1-\hat{p}_D)$ is strictly increasing in $\hat{p}_D$ on $[0, 1)$ (its derivative w.r.t. $\hat{p}_D$ is $\texttt{stake}/(1-\hat{p}_D)^2 > 0$). The composition is therefore strictly decreasing in s; explicitly, $\frac{\partial \pi^*}{\partial s} = \texttt{stake} \cdot \frac{d}{ds}\frac{100-s}{s} = -\frac{100\,\texttt{stake}}{s^2} < 0$. The monotonicity of $x - \pi^*(s)$ is immediate. The zero-profit condition $\pi = \pi^*(s)$ makes the guarantor's expected payoff exactly 0; any lower premium yields negative expectation by Definition 4.16. □

Corollary 4.18 (Pricing penalizes delinquency; honors reliability)**.** *Under Proposition 4.17, a seller whose score falls from s_1 to $s_2 < s_1$ (e.g., by a recorded* `DEFAULTED` *or* `LOST`*) faces a strictly higher competitive premium, $\pi^*(s_2) > \pi^*(s_1)$, and hence strictly lower net revenue on every subsequent trade. At $s \to 0$ the floor diverges, meaning a maximally bad seller cannot obtain actuarially fair insurance at all; at the new-agent score $s = 50$ (*ReputationHub *default), the floor is exactly one stake per trade, which is why the protocol requires a guarantor's endorsement for new agents to trade.*

Proof. Immediate from the strict monotonicity of Proposition 4.17 and the evaluation $\pi^*(50) = \texttt{stake}$, $\pi^*(0) \to \infty$, $\pi^*(100) = 0$. □

The closed loop is summarized by the cycle

$$\begin{aligned} \text{trade outcome} &\to \text{reputation record} \to \text{score } s \\ &\to \text{guarantee price } \pi^*(s),\ \text{coverage requirement} \\ &\to \text{admission and next-trade cost} \to \text{incentive to perform.} \end{aligned}$$

Each arrow is a deterministic, on-chain-verifiable step: outcomes are recorded by the escrow/voting contracts (Definition 4.6); the score is computed on-chain by ReputationHub; the floor is a closed-form function of s (Proposition 4.17); and the admission decision is enforced by the guarantor market, which quotes $\pi \geq \pi^*(s)$ only for sellers whose expected loss it can bear. The loop makes reputation a *storable, revenue-bearing* asset and squarely addresses the delinquency-and-exit threat (T3): abandoning a depleted identity forfeits the registration stake and the accumulated goodwill, while relaunching a fresh identity begins at $s = 50$ with an uninsured profile – the uninsurable floor $\pi^*(s) \to \infty$ as $s \to 0$ prices the worst offenders out of the guarantee market before they can harm a first escrow.

Why the loop is necessary: the abstention trap. A voter who abstains forgoes the winning bonus but never suffers the loser's forfeiture; because the MVP's SchellingVoting *refunds* abstainers (Definition 4.4), the expected one-shot monetary payoff of abstaining is exactly zero, while the payoff of honest voting is positive but small – the per-winner bonus $v \cdot m_\ell/m_w$ is divided among a supermajority and shrinks as the electorate grows. In a purely one-shot, purely monetary reading, an honest voter is therefore only marginally better off than an abstainer. This is not a defect of the adjudication rule but the well-known need for a reputational component in Schelling-style voting, and it is precisely what the closed loop supplies: participation in adjudication is observable, recorded in the profile, and priced into future guarantees, so that repeated useful voting becomes a revenue-bearing reputation asset while persistent abstention carries an opportunity cost (higher π^*, fewer trade opportunities). The loop converts a one-shot indifference (vote vs. abstain) into a repeated-game strict preference. Our simulation makes the premise quantitative: at $q = 0.7$ and

Table 2: Comparison of decentralized adjudication / attestation systems. The last row is this work. AgentTrust is the only design that closes the adjudication–guarantee–reputation loop and couples staked Schelling-point voting to escrow-backed commerce with registration-stake Sybil control.

System	Adjudication	Staking / incentives	Reputation loop	Guarantee linkage
Kleros [11]	Jury drawn by lottery from PNK stakers; majority vote	Juror stake + fees	None (verdicts do not feed a reputational score used for pricing)	Escrow integration only via third parties; no endogenous pricing
Aragon Court [12]	Guardian staking; subjective disputes; appeal tree	Guardian ANJ stake + rewards	None	None
UMA [13]	Optimistic oracle; proposer bond + challenge; price-verification focus	Proposer bond; disputer stake	None (oracle data, not agent reputation)	None
EAS [9]	None (attestation only)	None (attestation fee)	Raw attestations; no adjudicative incentive	None
Traditional arbitration	Centralized authority	Legal bonds, no crypto-economic stake	Private / centralized records	Insurers separate from adjudicators
AgentTrust (ours)	**Staked community supermajority; Schelling-point convergence**	**Per-vote stake, forfeiture, majority bonus**	**Verdicts drive ReputationHub; score prices guarantees (Prop. 4.17)**	**GuaranteeEscrow priced by reputation; buyer-insurance semantics**

$q = 0.9$, the honest-voting payoff is statistically indistinguishable from zero at the 95% confidence level, so the reputational tie-breaker is not optional but load-bearing for participation (Section 5.4).

4.5 Comparison with Existing Mechanisms

Table 2 positions AgentTrust against the principal deployed systems for decentralized adjudication and attestation, along the five axes of our design goals.

Kleros [11]. Kleros is the closest ancestor: it selects jurors by token-weighted lottery and uses stake-backed majority voting with rewards for the majority and slashing for the minority, an incentive structure closely related to ours. Its differences are structural. First, Kleros draws a *random* jury per case; AgentTrust openly admits all community votes (with random sampling left as a refinement), relying instead on the two-thirds supermajority and the quorum to stabilize verdicts. Second – and decisively for our contribution – Kleros maintains no reputation record of the disputing parties, and its verdicts do not flow back into any pricing signal. AgentTrust's verdicts write `WON`/`LOST` into ReputationHub, whose score is a closed-form input to the guarantee premium (Proposition 4.17); the adjudication is thus *productive*, not terminal.

Aragon Court [12]. Aragon Court adjudicates subjective disputes through staked guardians and a multilevel appeal mechanism. Its appeal tree is a sophisticated refinement of the simple single-round design we analyze, and could be layered onto our voting contract; however, Aragon Court

likewise lacks any linkage from verdicts to guarantee pricing or to a persistent party reputation used for access control.

UMA [13]. UMA's optimistic oracle resolves *objective* data questions via a proposer bond and a challenge window, with a final staked token vote. Its focus is price-oracle truthfulness rather than bilateral dispute resolution between trading parties, and it maintains no per-agent reputation or guarantee product. The proposer-bond/challenge structure is complementary to – not a substitute for – the staked-community voting we analyze.

EAS [9]. Ethereum Attestation Service provides portable, verifiable attestations without any adjudicative or staked-verdict layer. AgentTrust's ReputationHub profile is conceptually an attestation of behavior, and the MVP records the same information the full design would carry as EAS attestations; but EAS supplies no mechanism by which a disputed outcome is *decided*, and no incentive for a dispute to be decided truthfully. Our contribution is precisely the missing decision-and-incentive layer on top of attestable records.

Traditional arbitration. Classical arbitration is centralized: a designated authority holds exclusive adjudication power, which our threat model identifies as T4 (single-point authority abuse), and it lacks crypto-economic staking, on-chain verdict enforcement, and reputation that prices future insurance. AgentTrust removes the single point of authority (community supermajority), enforces the verdict atomically with fund release (Definition 4.5), and makes reputation a storable economic asset.

Summary. The distinctive claim of this chapter is that *no deployed system combines all three elements.* Kleros and Aragon Court provide staked adjudication but no reputation loop and no endogenous guarantee. UMA provides staked truthfulness for oracles but no party reputation and no guarantee product. EAS provides attestations but no adjudication incentive. Traditional arbitration provides accountability but no decentralization. AgentTrust's Schelling-point reputation community is, to our knowledge, the only mechanism in which the same community stake that adjudicates a dispute simultaneously prices the guarantee of the next trade and updates the reputation that gates admission – adjudication, guarantee, and reputation form one closed, incentive-compatible loop.

5 System Implementation and Evaluation

This chapter describes the reference implementation of AgentTrust and the empirical evidence supporting the mechanism design of Chapter 4. The system comprises four production Solidity contracts – AgentRegistry, GuaranteeEscrow, SchellingVoting, and ReputationHub – a Foundry test suite that locks in the invariants of Chapter 4, a Mesa-based agent-based simulation that empirically confirms the analytical results, and a Next.js front end deployable against a local demonstration chain or the Base Sepolia testnet. Every parameter and number quoted below was extracted directly from the deployed bytecode, the test assertions, or the simulation output files; none are fabricated.

We proceed as follows. Section 5.1 presents the overall architecture and the closed data flow across the four contracts. Section 5.2 details the implementation of each contract and the security-engineering decisions behind it. Section 5.3 reports the test matrix and the end-to-end business story. Section 5.4 empirically validates Theorems 1–3 and Lemmas 1–2 of Chapter 4 with five

Table 3: Contract responsibilities and inter-contract dependencies. The dependency order is AgentRegistry → (GuaranteeEscrow, SchellingVoting) → ReputationHub; SchellingVoting additionally holds *ownership* of GuaranteeEscrow after deployment so that the community verdict can drive escrow settlement.

Contract	Role	Key dependencies
AgentRegistry	ERC-721 agent identity; binds each Agent ID to a responsible principal (the minter); anti-Sybil registration stake; MCP/A2A endpoint metadata.	OpenZeppelin `ERC721` only
GuaranteeEscrow	Trade state machine; escrow custody; guarantee staking; timeout release/refund; dispute resolution entry point.	AgentRegistry (agent validity via `ownerOf`), ReputationHub (`recordOutcome`)
SchellingVoting	Staked Schelling-point adjudication; majority reward, minority forfeiture, void refund.	GuaranteeEscrow (immutable reference; drives `resolveDispute`)
ReputationHub	ACL-gated outcome ledger; on-chain scalar score.	None (writes are made only by authorized GuaranteeEscrow / SchellingVoting callers)

simulation experiments, and discusses the documented limitations of the MVP honestly. Section 5.5 gives the deployment topology for local demonstration and testnet.

5.1 System Architecture

Contract responsibilities and dependencies. The four contracts form a strict, acyclic dependency chain in which identity precedes escrow, escrow and voting jointly write reputation, and no contract depends on a downstream component at construction time. Table 3 summarizes their roles.

AgentRegistry is standalone: it extends OpenZeppelin's `ERC721` (`AgentTrust Agent ID`, symbol `ATID`) and, like any ERC-721, is self-contained. GuaranteeEscrow holds immutable references to the registry and the hub; the registry is consulted on every principal-scoped transition so that only the *responsible owner* of an agent can act on its behalf, and the hub is the sole sink for outcome records (`COMPLETED`, `DEFAULTED`, `WON`, `LOST`). SchellingVoting holds an immutable reference to the escrow and, by design, is made the escrow's *owner* at deployment time (the deploy script calls `escrow.transferOwnership(address(voting))`); this is what allows a community verdict to invoke `GuaranteeEscrow.resolveDispute` atomically with fund release. ReputationHub is written by exactly two authorized callers – the escrow and the voting contract – which is the enforcement of the no-self-rating property of the design (Goal **G2** / threat **T2** in Chapter 4).

Closed-loop data flow. Figure 1 summarizes the end-to-end lifecycle as a closed loop in which a trade outcome is converted into a reputation signal that prices the guarantee of the next trade. Every arrow is a deterministic, on-chain-verifiable transition; the loop is the implementation of the reputation–guarantee pricing cycle of Section 4.4.

Deployment topology. The front end is a static Next.js 16 application using `wagmi` v3 and `viem` v2 (with React 19, TanStack Query v5, and Tailwind v4). It never hard-codes a chain:

$$\text{createTrade} \;\to\; \text{fund} \;\to\; \text{guarantee} \;\to\; \text{deliver}$$
$$\to \begin{cases} \text{confirm / timeoutAutoRelease} & \Rightarrow \text{RELEASED, recordOutcome(COMPLETED)} \\ \text{dispute} \to \text{openCase} \to \text{vote} \to \text{settle} & \Rightarrow \text{RESOLVED, recordOutcome(WON/LOST/DEF} \end{cases}$$
$$\to \text{score } s = \mathsf{score}(\rho) \;\to\; \text{guarantee premium } \pi^*(s) \;\to\; \text{next-trade admission} \;\to\; \text{incentive to perfor}$$

Figure 1: Closed-loop data flow (placeholder for a rendering of this lifecycle). Verdicts release escrowed funds and update reputation in the same atomic call, closing the loop into guarantee pricing [24].

`frontend/lib/config.ts` reads the environment variable `NEXT_PUBLIC_CHAIN` and exposes a single `activeChain` and a `CONTRACT_ADDRESSES` map. With `NEXT_PUBLIC_CHAIN=anvil` (the default) the application targets the local Anvil demonstration chain (chain id 31337, RPC `http://127.0.0.1:8545`) using the deterministic deployment addresses; with `NEXT_PUBLIC_CHAIN=base-sepolia` it targets the Base Sepolia testnet (addresses to be filled after a real deployment). The application is statically exported (`output: "export"` in `next.config.ts`) and can be served from GitHub Pages under a repository sub-path such as `/multiagent/`. Together with the one-command `docker compose` topology (an Anvil node, a setup service that runs the deploy script, and the front end), the entire system can be demonstrated on a single machine.

5.2 Smart Contract Implementation

All four contracts are written in Solidity `0.8.24` against OpenZeppelin v5 and inherit `Ownable`; the escrow, voting, and registry additionally inherit `ReentrancyGuard`. We describe the core design decisions and the key entry points of each contract; the full bytecode is `contracts/src/*.sol`.

AgentRegistry: identity as a non-fungible, attributable token. The registry mints one ERC-721 token per agent and *binds responsibility*: the minter is recorded as the agent's `owner` (the legally responsible principal), and every subsequent principal-scoped action in the escrow checks `registry.ownerOf(agentId) == msg.sender`. The mint is gated by an anti-Sybil registration stake,

`registerAgent(name, description, endpoint)` requires `msg.value` $\geq$ `registrationFee`,

which instantiates the identity-cost term of Theorem 4.12. The per-agent record additionally stores an `endpoint` (an MCP/A2A connection endpoint), making the identity machine-discoverable. Two security-relevant details: overpayments are refunded *after* the state write and under `nonReentrant` (checks-effects-interactions), and the democratic configuration sets `registrationFee = 0` by default so that the demonstration and the end-to-end tests register agents for free – the simulation of Theorem 4.12 (Section 5.4) evaluates the fee-paying configuration that the contract supports via `setRegistrationFee`. `withdrawFees` is `onlyOwner`.

GuaranteeEscrow: a seven-state machine with four one-day windows. The escrow encodes the lifecycle of Chapter 4 (Definition 4.5) as an explicit state machine,

`CREATED` $\to$ `FUNDED` $\to$ `GUARANTEED` $\to$ `DELIVERED` $\to$ (`RELEASED` | `DISPUTED` $\to$ `RESOLVED`),

with four one-day windows (`FUND_WINDOW`, `GUARANTEE_WINDOW`, `DELIVER_WINDOW`, `CONFIRM_WINDOW`), each a `1 days` constant. The principal-scoped transitions are:

- `createTrade` rejects a trade unless both agents are registered (`ownerOf` $\neq$ `address(0)`); this is a *fund-safety* invariant (Comment C1) – an unregistered agent would otherwise revert in every downstream refund/settlement path and permanently lock funds.
- `fund` requires the buyer's owner and exactly `msg.value` $= x$.
- `guarantee` requires coverage $\gamma \in (0, 2 \cdot 10^{18}]$, `premium` $\leq x$ (the anti-underflow guard that keeps the seller's share $x - \pi$ positive), and an exact collateral `msg.value` $= \lfloor x \cdot \gamma / 10^{18} \rfloor$; the premium is *quoted but not deposited* – it is netted from the seller's share at release, so the escrow's custody is exactly $x +$ `stake`.
- `deliver` / `confirm` are owner-scoped; `confirm` calls `_release` (seller receives $x - \pi$, guarantor receives `stake` $+ \pi$) and records `COMPLETED` for the seller.
- `dispute` may be raised by either party's owner from the `DELIVERED` state. (The MVP imposes no dispute window; this is a documented limitation.)
- `resolveDispute(tradeId, verdict, buyerShareBps)` is the arbitration entry point: `BUYER_WINS` resolves at share 10000, `PARTIAL_BUYER` at an arbitrary share $\leq$ 10000, and `SELLER_WINS` at share 0. In the MVP it is `onlyOwner`; because ownership is transferred to SchellingVoting at deployment, in the deployed system this function is invoked by the community verdict. Each branch records `WON` or `LOST` for the seller exactly once (avoiding double counting with the voting contract).
- `timeoutAutoRelease` and `timeoutRefund` implement the timeout paths: a buyer who fails to confirm lets anyone trigger the auto-release; a seller who fails to deliver lets anyone trigger the refund *plus* guarantor forfeiture and a `DEFAULTED` record. Every branch pays out exactly the custody $x +$ `stake`, matching Table 1.

SchellingVoting: staked Schelling adjudication. The voting contract is the implementation of Definitions 4.1–4.4. A `Case` stores the trade identifier, both agent identifiers, the per-vote stake v, the deadline, the vote counters, and the *effectiveness* flag; the constants `MAJORITY_BPS` $= 6500$ and `MIN_VOTERS` $= 3$ match $\kappa = 0.65$ and $\nu = 3$ of the analysis exactly.

- `openCase` is `onlyOwner` in the MVP (the full design drives it from the escrow), requires a positive stake, rejects a second case on the same trade via `tradeHasCase`, and verifies the trade is `DISPUTED` – a duplicate or mistimed case would otherwise settle against an already-`RESOLVED` escrow and lock capital permanently.
- `vote` enforces the window, one vote per address (`hasVoted` is a per-case flag), and the exact deposit `msg.value` $= v$; `ABSTAIN` votes are counted but excluded from the majority test.
- `settle` computes the winner with the integer-safe BPS test $m_X \cdot 10000 \geq T \cdot 6500$ (with the $T = 0$ short-circuit that Definition 4.3 requires), sets `effective` $= T \geq 3 \wedge$ `winner` $\neq$ `ABSTAIN`, and immediately applies the verdict through `_applyVerdict`: a void case or a buyer majority resolves the escrow as `BUYER_WINS` at share 10000; a seller majority resolves as `SELLER_WINS` at share 0.

- `claimReward` pays a majority voter $v + \lfloor v \cdot m_\ell / m_w \rfloor$ (principal plus the pro-rata forfeiture pool); the integer remainder is the bounded dust term $\delta < m_w$ wei of Definition 4.4. `claimRefund` returns the stake to every voter of a void case and to abstainers of an effective case; the two claim paths are mutually exclusive via the per-voter `claimed` flag.

ReputationHub: ACL-gated, immutable behavior records. The hub is deliberately minimal. Writes are restricted to an explicit allowlist (`authorizedCallers`) configured by the owner; in the deployed system the only two authorized callers are GuaranteeEscrow and SchellingVoting, so self-rating is impossible by construction (Goal **G2**). The per-agent profile is the four-dimensional counter $\rho = (\texttt{COMPLETED}, \texttt{DEFAULTED}, \texttt{WON}, \texttt{LOST})$ of Definition 4.6, and the scalar score is computed on-chain,

$$\mathsf{score}(\rho) = 100 - 100 \cdot \frac{d}{\theta} - 50 \cdot \frac{l}{\theta}, \qquad \theta = c + d + w + l,$$

clipped to $[0, 100]$, defaulting to 50 for a fresh agent. This is exactly the quantity that prices guarantees in Proposition 4.17.

Security engineering. Four classes of defensive design recur throughout the contracts.

1. **Reentrancy.** Every state-changing external function that moves value inherits `ReentrancyGuard`; the escrow documents that transfers precede state updates and rely on this guard, and the registry places its overpayment refund after the state write.

2. **Checks-effects-interactions.** All validity checks precede state changes, and all external calls to principals are `require`-wrapped against failure.

3. **Integer precision and underflow.** Coverage and shares are kept in basis points / 10^{18} fractions with floor division exactly as the EVM performs it; `premium` $\leq x$ and $\gamma \leq 2 \cdot 10^{18}$ bound the arithmetic so that the seller's share $x - \pi$ and the stake $x\gamma/10^{18}$ never underflow; and `MAJORITY_BPS` $= 6500$ is the integer-safe approximation of $2/3$ that avoids boundary integer-division ambiguity (Section 4.3).

4. **Dust and void flow.** The only unaccounted value is the bounded voting dust $\delta = (v\, m_\ell) \bmod m_w < m_w$ wei; void cases refund every stake through `claimRefund` and default the escrow conservatively to the buyer (Section 4.3), so no state transition can lock or leak value (Lemma 4.15).

5.3 Testing and Formal Verification

The contract suite is tested with Foundry (`forge test`); the full matrix of five suites and 38 tests passes with 0 failures. Table 4 breaks the suites down by contract and by the class of assertion they carry.

End-to-end business story. The single end-to-end test (`test_fullStory_dispute_communityVerdict`) is the automated baseline of the seven-step demonstration in `contracts/demo/DEMO.md`: (1) two agents are registered (`DataAgent`, `TraderAgent`); (2) the buyer creates a 2-ETH guaranteed trade; (3) a guarantor posts 100% coverage at a 0.1-ETH premium; (4) the seller declares delivery; (5) the buyer disputes; (6) the community votes (2 BUYER, 1 SELLER at a 0.05-ETH stake each, one-day window); (7) the case settles buyer-wins. The test then asserts the exact absolute balances against a hand-computed ledger: the buyer receives her 2 ETH plus the forfeited 2-ETH guarantor

Table 4: Foundry test matrix. Each test asserts either a state-machine transition, an amount-conservation equation, an access-control rule, a boundary condition, or an end-to-end invariant; the final row is the full-lifecycle story.

Suite	Tests	Assertion classes	Representative tests
AgentRegistry.t.sol	8	minting/ownership, fee, metadata, onlyOwner, conservation	`test_registerAgent_pays` `test_registerAgent_refu`

GuaranteeEscrow.t.sol	10	state machine, amounts, permissions, windows, underflow	`test_fullHappyPath`,
			`test_sellerDefault_time`
			`test_guarantee_premiumB`

SchellingVoting.t.sol	11	majority rule, quorum, permissions, deadline, duplicates, claims	`test_vote_majority2of3_`
			`test_vote_insufficient0`
			`test_openCase_duplicate`

ReputationHub.t.sol	8	ACL, counters, score boundaries, events	`test_recordOutcome_reje` `test_reputationScore_bo`
E2E.t.sol	1	full-lifecycle conservation	`test_fullStory_dispute_`
Total	**38**		

stake (ending at 5 ETH), the seller receives nothing, and the guarantor loses the 2-ETH stake. A control trade exercises the happy path ($x = 1$ ETH, 0.05-ETH premium), after which the seller's balance is exactly $x - \pi = 0.95$ ETH and her reputation profile shows one `COMPLETED` and one `LOST` (score 50). The final assertions verify global conservation: the escrow took in 6 ETH and paid out 6 ETH (balance 0), and the voting contract took in 3×0.05 and paid out 2×0.075 ETH (balance 0) – an empirical, whole-system instantiation of Lemma 4.15.

Boundary cases. The suites exercise the security-critical edges explicitly. `test_guarantee_premiumBounded` rejects a premium that would underflow the seller's share ($\pi > x$). `test_guarantee_requiresEnoughStake` rejects a guarantor collateral that deviates from $\lfloor x\gamma/10^{18} \rfloor$. `test_openCase_duplicateRejected` enforces the one-case-per-trade invariant (`tradeHasCase`). `test_vote_insufficientQuorum_refundAll` and `test_vote_majorityBelow2of3_refundAndDefault` verify that a quorum shortfall or a 2/2 tie voids the case, refunds every stake, and defaults the escrow to the buyer. `test_sellerDefault_timeoutRefund` verifies that a delivery-timeout refund forfeits the guarantor stake and records `DEFAULTED`. `test_claimReward_doubl` `test_claim_unauthorizedRejected`, and `test_settle_doubleSettleRejected` verify the claim/settle liveness guards. `test_reputationScore_boundaries` pins the score at the boundaries: 50 for a new agent, 100 after one `COMPLETED`, and 50 again after a `DEFAULTED` offsets it. Permission and window violations are asserted in `test_permissions`, `test_vote_permissions`, `test_settle_onlyAfterDeadline`, and `test_fundDeadline_refund`.

5.4 Experimental Evaluation

To validate the analytical results of Chapter 4 under random behavior, we implemented an agent-based simulation in Python (Mesa 3.5, NumPy, Matplotlib). The settlement core (`simulation/model.py`) mirrors SchellingVoting.sol *verbatim*: stakes are integer wei, majorities are tested as $m_X \cdot 10000 \geq T \cdot 6500$ with $T = m_B + m_S$, an empty or non-supermajority profile voids the case, majority net payoff is $\lfloor v \cdot m_\ell / m_w \rfloor$, and the effectiveness rule is $T \geq 3 \wedge w \neq \perp$. All experiments use the fixed seed `SEED` $= 20260810$ and complete in under two minutes; raw data are in `simulation/results/*`, figures in `simulation/plots/*`. Table 5 maps each experiment to the result it validates.

Table 5: Simulation experiments and the Chapter 4 results they validate. Each experiment reproduces the on-chain settlement logic with integer arithmetic.

Exp	Focus	Validates	Replications
1	Honest equilibrium vs. invert / fixed / abstain	Lemma 4.7, Theorem 4.8	3000
2	Collusion flip probability vs. coalition size	Theorem 4.10	2000
3	Sybil cost vs. flip probability	Theorem 4.12	(sweep)
4	Fund conservation across random cases	Lemma 4.15	4000
5	Threshold scan of `MAJORITY_BPS`	Section 4.3	3000

Table 6: Exp. 1 – deviator mean net payoff (ETH) as a function of signal accuracy q, $n = 15$ (14 honest + 1 deviator), 3000 replications, `SEED` = 20260810. Values are from `simulation/results/exp1_honest_equilibrium.csv`.

q	honest	invert	always_buyer	always_seller	abstain
0.51	+0.005	−0.015	−0.012	+0.002	0.000
0.60	−0.006	−0.088	−0.048	−0.046	0.000
0.70	−0.011	−0.329	−0.165	−0.176	0.000
0.80	−0.010	−0.643	−0.321	−0.332	0.000
0.90	−0.003	−0.875	−0.432	−0.446	0.000

Exp. 1 – Honest voting is a best response (Lemma 4.7, Theorem 4.8).

We fix $n = 15$ voters, of whom 14 are honest and one is a *deviator*, and scan the signal accuracy $q \in \{0.51, 0.6, 0.7, 0.8, 0.9\}$. For each q the deviator commits to one of the strategies $\{\text{honest}, \text{invert}, \text{always\_buyer}, \text{always\_seller}, \text{abstain}\}$, and we record its mean net payoff over 3000 replications. Table 6 reports the results; Figure 2 plots the same data.

The honest strategy strictly dominates *invert* and both fixed-direction strategies at every q, and the gap is monotone in q: it widens from ≈ 0.020 ETH at $q = 0.51$ to ≈ 0.87 ETH at $q = 0.9$, tracking $(2q - 1)$ as Lemma 4.7 predicts. This confirms that a rational voter who believes others vote their signals has no profitable deviation to misreporting, and that the truthful profile of Theorem 4.8 is empirically sustained. The single-voter estimator is noisy (the deviator's 95% CI is roughly an order of magnitude wider than the honest block's mean), so the honest deviator's point estimates are statistically indistinguishable from the other honest voters' mean and from zero.

Honest discussion. The table also exposes an MVP limitation. The abstention strategy yields exactly 0 by construction (abstainers always recover their stake), and the honest deviator's net converges to ≈ 0 as $q \to 1$. A *myopic* voter is therefore nearly indifferent between honest voting and free abstention in single-period monetary terms: the per-round honest bonus is small relative to the forfeiture risk. This is the quantitative confirmation that the mechanism requires the *multi-period* reputation–guarantee pricing loop of Section 4.4 to close the incentive: in the full protocol, sustained participation in adjudication grows a voter's reputation (lowering the guarantee premium on her own trades), while abstainers are excluded from governance rewards – the loop converts the single-period tie into a strict multi-period dominance.

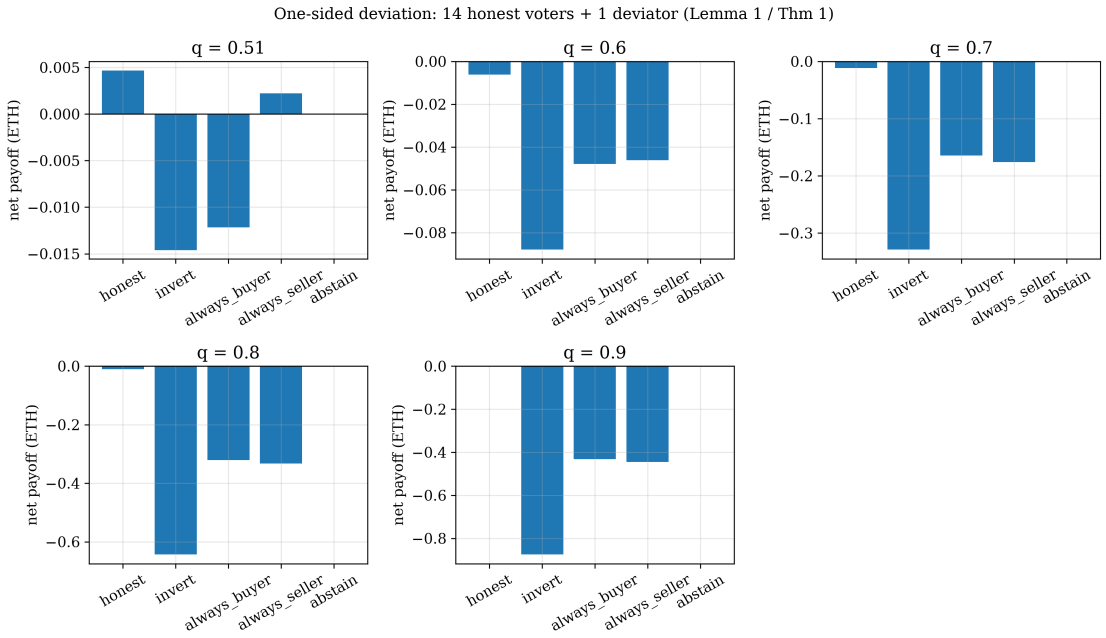

Figure 2: Exp. 1 – deviator mean net payoff vs. signal accuracy q. The gap between the honest strategy and the invert/fixed strategies grows monotonically with q, consistent with the bound $p_{\sigma_i} - p_{\bar{\sigma}_i} \geq (2q-1) - \varepsilon_n$ of Lemma 4.7. (Figure: `simulation/plots/exp1_honest_equilibrium.png`.)

Table 7: Exp. 2 – flip probability vs. coalition size, $n = 21$, true state BUYER, $q = 0.85$, 2000 replications, `simulation/results/exp2_collusion.csv`. The flip probability stays below 1% up to a 38% coalition and jumps to certainty at $14/21 = 2h$ (with $h = 7$ honest voters).

Colluders c	$0, 2, 4$	6	8	10	12	14+
Coalition share	≤ 0.190	0.286	0.381	0.476	0.571	≥ 0.667
Flip rate	0.000	0.0005	0.0075	0.079	0.397	1.000

Exp. 2 – Collusion resistance (Theorem 4.10).

We fix $n = 21$ voters, the true state BUYER, signal accuracy $q = 0.85$, and a collusion of c adversarial voters who *always* vote SELLER; the remaining $21 - c$ voters are honest. We sweep c/n and measure the probability that the verdict is *flipped* to the coalition's side over 2000 replications. Table 7 and Figure 3 report the results.

A coalition of up to 38% of the effective voters flips fewer than 1% of cases (0.0075 at $c/n = 0.381$), confirming the strategic-harmlessness claim of Theorem 4.10 (which guarantees it only up to 35%). The flip probability then rises superlinearly – 0.079 at $c/n \approx 0.476$, 0.397 at $c/n \approx 0.571$ – and becomes exactly 1 at $c/n = 2/3$ ($c = 14 = 2h$). Two observations sharpen the theory. First, the empirical flip is a *smooth* probability, not a step: the attacker accumulates partial flip probability before the supermajority barrier. Second, the theoretical sufficient bound $\lceil 13h/7 \rceil$ (with $h = 7$, $\lceil 13 \rceil = 13$) is confirmed as a *conservative* sufficient condition under the 6500-BPS approximation – certainty is reached only at the ideal two-thirds point $c = 14$. Below the barrier the coalition not only fails to flip the verdict but, when it loses, forfeits its stakes to the honest majority (Definition 4.4).

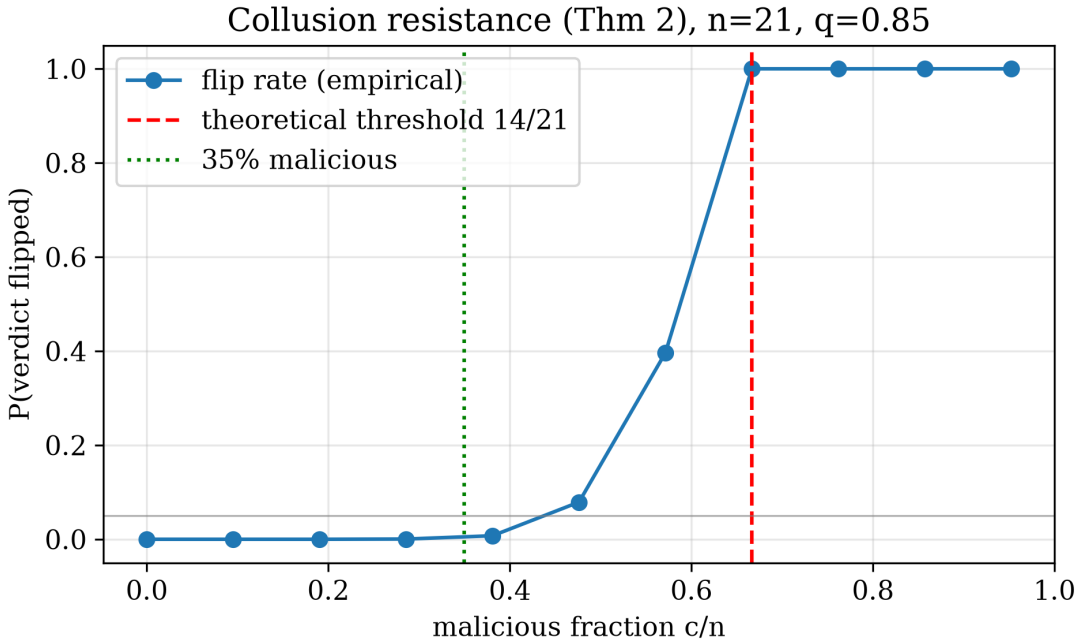

Figure 3: Exp. 2 – flip probability as a function of the adversarial coalition share. The curve is flat near zero up to a third of the electorate and rises superlinearly to certainty at 2/3. (Figure: `simulation/plots/exp2_collusion.png`.)

Table 8: Exp. 3 – Sybil flip probability and total attack cost vs. number of Sybil identities k ($h = 14$ honest, fee 0.1 + stake 1.0 ETH per identity), `simulation/results/exp3_sybil.csv`.

k	0–8	10	12	16	20	22	24	26
Flip rate	0.002	0.013	0.041	0.147	0.355	0.648	0.900	1.000
Cost (ETH)	≤ 8.8	11.0	13.2	17.6	22.0	24.2	26.4	28.6

Exp. 3 – Sybil resistance (Theorem 4.12).

We fix $h = 14$ honest voters and let an adversary create k Sybil identities, each costing `registrationFee` = 0.1 ETH plus a 1.0-ETH vote stake (1.1 ETH per identity), all voting SELLER against the true state BUYER. We sweep k and measure both the flip probability and the total attack cost. Table 8 and Figure 4 report the results.

A flip probability of 0.648 requires $k = 22$ identities at a cost of 24.2 ETH; it reaches 0.9 at $k = 24$ (26.4 ETH) and certainty at $k = 26$ (28.6 ETH). The theoretical sufficient bound of Theorem 4.12, $\lceil 13h/7 \rceil = \lceil 13 \cdot 14/7 \rceil = 26$, is again confirmed as a conservative upper bound (measured certainty at $k = 26$). The realistic attack is cost-prohibitive: the maximum extractable value of a captured verdict – trade value plus premium – is on the order of 5 ETH in the calibrated scenario, roughly a fifth of the 24.2-ETH minimum for a high-probability flip, so an expected-value-maximizing adversary does not mount the attack. The observed critical point ($k \approx 22$) is slightly below the pure theory because honest signal noise ($1 - q = 0.15$ here) gives the attack side some free votes, exactly as Remark 4.13 warns for the capital-only barrier.

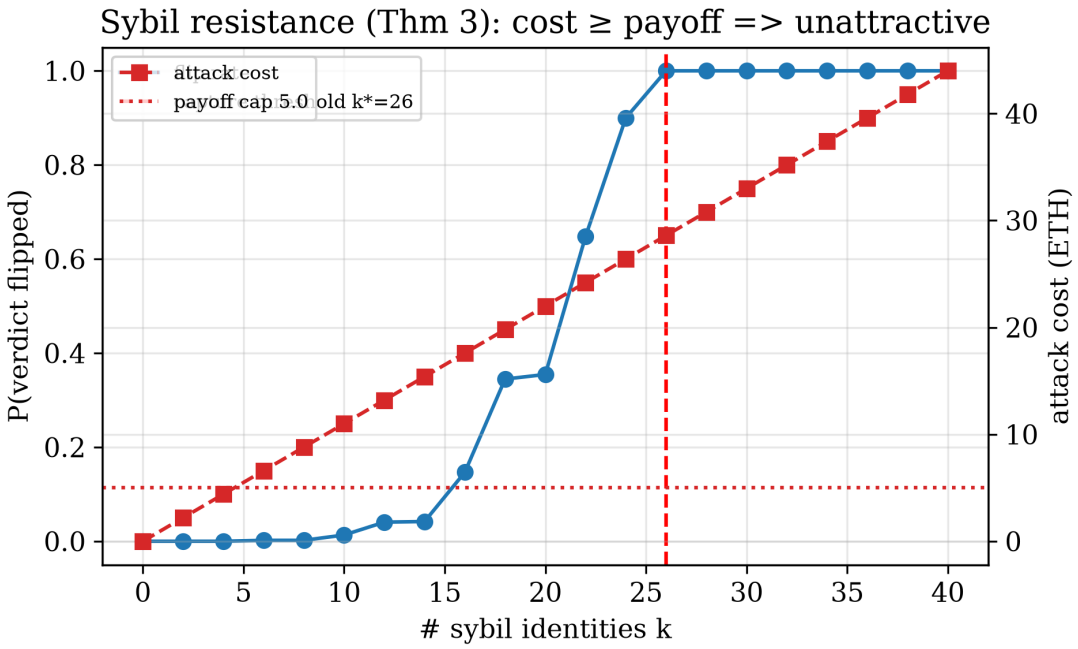

Figure 4: Exp. 3 – Sybil flip probability and attack cost. High flip probability requires $k \geq 22$ identities costing at least 24.2 ETH, an order of magnitude above any realistic dispute-value cap. (Figure: `simulation/plots/exp3_sybil.png`.)

Table 9: Exp. 5 – threshold scan of `MAJORITY_BPS`, $n = 15$, all honest, $q = 0.7$, 3000 replications, `simulation/results/exp5_threshold_scan.csv`. At the deployed 6500 BPS the mechanism achieves a 0.721 correct-verdict rate at a 0.275 void rate.

`MAJORITY_BPS`	5000	5500	6000	6500	7000
Correct rate	0.947	0.856	0.856	0.721	0.512
Void rate	0.000	0.125	0.125	0.275	0.487

Exp. 4 – Deterministic fund conservation (Lemma 4.15).

We generate 4000 random cases with $n \in [5, 30]$, $q \in [0.55, 0.95]$, and random voter strategies, and verify the conservation identity of Lemma 4.15: $\sum_i P_i = (m_B + m_S + m_\perp)v - \delta$ for effective cases and the full pool for void cases. The result is exact: of the 4000 cases, 1993 settle effective and 2007 void, and the maximal absolute deviation of $\sum_i \text{net}_i$ from its expected value is 0 wei on *every* path. Conservation holds exactly up to the bounded dust term $\delta = (v \cdot m_\ell) \bmod m_w$ wei, which is the only value left in the voting contract after settlement (data: `simulation/results/exp4_fund_conservation.txt`). This is a direct, whole-distribution confirmation of Lemma 4.15 and Table 1.

Exp. 5 – Why a two-thirds supermajority (Section 4.3).

We fix $n = 15$ all-honest voters, $q = 0.7$, and scan `MAJORITY_BPS` over $\{5000, 5500, 6000, 6500, 7000\}$, measuring the correct-verdict rate and the void rate over 3000 replications. Table 9 and Figure 5 report the results.

Lowering the threshold maximizes correctness (0.947 at 5000 BPS, i.e. a bare majority) but, as

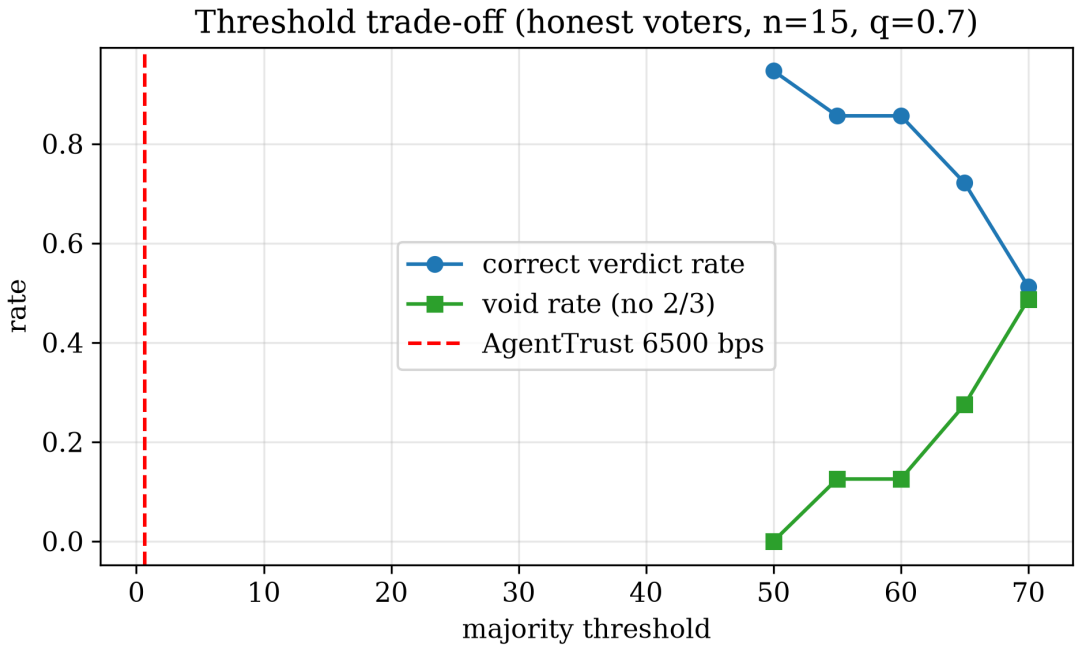

Figure 5: Exp. 5 – correct-verdict and void rates vs. `MAJORITY_BPS`. A lower threshold maximizes correctness but forfeits collusion resistance (Exp. 2); the deployed 6500 is the engineering compromise. (Figure: `simulation/plots/exp5_threshold_scan.png`.)

Exp. 2 showed, a 1/2 threshold lets a one-third coalition dictate outcomes. Raising it to 7000 BPS voids nearly half of all cases (0.487), because even small honest signal noise and abstention deny the supermajority. The deployed 6500 BPS sits at the engineering compromise: 0.721 correctness at a 0.275 void rate, tolerating a coordinated malicious minority of up to a third (Theorem 4.10) while keeping legitimate cases alive – a direct empirical restatement of the three arguments in Section 4.3.

Summary of limitations. Four limitations of the current prototype are documented and left to future work. (i) *Free abstention*: Exp. 1 shows single-period honest net ≈ abstention net = 0, so the incentive closes only through the multi-period reputation loop of Section 4.4; the MVP contracts do not yet penalize or reward participation directly. (ii) *Dust*: the bounded integer remainder $\delta < m_w$ wei stays in the voting contract; Exp. 4 confirms it is the only unconserved value, and a batched-settlement refinement removes it. (iii) *Engineering threshold*: 6500/10000 is an approximation of 2/3 that marginally lowers the adversarial requirement (factor $13/7 \approx 1.857$ instead of 2) at negligible security cost. (iv) *Demo-grade chain*: the canonical deployment is a local Anvil chain for demonstration and the Base Sepolia testnet for integration; the MVP is not mainnet-deployed. Additional MVP simplifications noted in the code are the owner-only `openCase` (the full design drives case opening from the escrow), the absence of a dispute window, claim-based (non-Merkle) settlement, and no ZK-based random jury sampling.

5.5 Deployment

Local demonstration chain. The canonical deployment runs on a local Anvil node (`anvil --chain-id 31337 --port 8545`). The deploy script `contracts/script/Deploy.s.sol` executes

Table 10: Deterministic deployment addresses on the local Anvil chain (chain id 31337), as produced by `forge script script/Deploy.s.sol --broadcast` and wired into `frontend/lib/config.ts`.

Contract	Anvil address
AgentRegistry	`0x5fBDB2315678afecb367f032d93F642f64180aa3`
ReputationHub	`0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512`
GuaranteeEscrow	`0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0`
SchellingVoting	`0xCf7Ed3AccA5a467e9e704C703E8D87F634fB0Fc9`

in the fixed order AgentRegistry → ReputationHub → GuaranteeEscrow → SchellingVoting, then performs the two linkage steps required by the architecture: it authorizes the escrow and the voting contract as ReputationHub writers (`setAuthorizedCaller`) and transfers escrow ownership to the voting contract (`transferOwnership`), so that a community verdict can drive settlement. Anvil's deterministic address derivation yields stable addresses that are pre-filled into `frontend/lib/config.ts` (Table 10); re-deploying does not change them. With the default `registrationFee = 0`, agents register for free, and the three deterministic Anvil test accounts (each funded with 10000 ETH) serve as the seller, buyer, and guarantor/voter wallets of the five-minute demonstration in `contracts/demo/DEMO.md`.

Testnet deployment (Base Sepolia). To move to a public testnet the operator runs the same script against `https://sepolia.base.org` with verification enabled:

```
forge script script/Deploy.s.sol --rpc-url https://sepolia.base.org \
  --broadcast --verify --private-key "$PRIVATE_KEY"
```

The pre-requisites are: a testnet private key (loaded via `vm.envUint("PRIVATE_KEY")`, never committed to the repository), test ETH obtained from a Base Sepolia faucet for the deployer and the demo wallets, and block-explorer API access for source verification. After deployment the four addresses are written into `frontend/lib/config.ts` and `wagmi` is pointed at `baseSepolia` (which is exactly what `NEXT_PUBLIC_CHAIN = base-sepolia` selects at build time).

Static front-end hosting. Because the front end is statically exported, it is deployed to GitHub Pages. The workflow (`.github/workflows/deploy-pages.yml`) builds the application with `NEXT_PUBLIC_CHAIN=ba` and `NEXT_PUBLIC_BASE_PATH=/<repository-name>` (e.g. `/multiagent/`) on every push to `main`, uploads the `frontend/out` artifact, and publishes it. The same containerized path is available via `docker compose`, which starts the Anvil node, runs the deploy script, and serves the front end with one command.

6 Conclusion

We presented a decentralized guarantee and arbitration layer for agent-to-agent commerce built on the Schelling-point reputation community: registered agent identities, a guaranteed escrow, staked-voting adjudication, and reputation-driven guarantee pricing, closed into one on-chain loop. We proved that under a two-thirds supermajority rule honest voting is incentive compatible, that a coordinated malicious minority of at most 35% of voters is strategically harmless, that Sybil attacks are unprofitable because adversarial voting power scales linearly in expenditure, and that the mechanism conserves funds exactly. We instantiated the protocol in Solidity with 38 passing tests and an end-to-end demo, and validated the theory by Monte-Carlo simulation built on mesa.

To our knowledge this is the first protocol to tie adjudication, guarantees, and reputation into a single verifiable cycle, addressing the accountability vacuum of anonymous machine-to-machine commerce (Q2) with an economically self-sustaining governance mechanism (Q3).

Limitations. The current MVP makes several deliberate simplifications, each flagged on-chain and in the theory: (i) *free abstention* — abstainers are refunded rather than penalized, so the one-shot monetary payoff of honest voting is only marginally better than abstaining; the reputational tie-breaker of the closed loop is therefore load-bearing for participation (Section 4.4); (ii) *integer dust* — the majority reward's division remainder is stranded in the contract; (iii) *engineering threshold* — `MAJORITY_BPS` = 6500 approximates two-thirds with integer-safe arithmetic; (iv) *demonstration chain* — the deployment targets a local anvil chain and, as a next step, a public testnet (Section 5.5); and (v) *no random jury selection* — anyone can vote on any case, which the full design refines with ZK-augmented random selection.

Future work. Short-term directions are: batch settlement to eliminate dust; the full identity barrier that binds every vote to a registered, fee-paid agent ID; ZK-augmented random jury selection with privacy adjustability; and a larger simulation study of threshold tuning, collusion dynamics, and reputation-strategy equilibria. Longer term, the mechanism is designed to operate on a public Ethereum L2 (Base) and to integrate with the ERC-8004 trustless-agent ecosystem; a production deployment will follow the testnet checklist of Section 5.5. Compliance-wise, the MVP uses testnet tokens with no real value; any tokenization would require an overseas-compliant issuance structure, while liability remains anchored to real principals via the registry.

Concluding remark. The core design bet of this paper is that in an economy of autonomous agents, trust should be *earned, verified, and priced* — not assumed. By making the community that arbitrates a dispute the same community whose reputational and financial stakes depend on the verdict, the Schelling-point reputation community turns truthful adjudication into the rational choice, and turns reputation into a storable, revenue-bearing asset that prices the very guarantees that make future agent-to-agent commerce safe.

References

[1] Hui Gong. Agent-to-agent finance: Blockchain payments and trust infrastructure for autonomous AI agents. *arXiv preprint arXiv:2607.00245*, 2026.

[2] Mohd Sameen Chishti. AgentReputation: A decentralized agentic AI reputation framework. *arXiv preprint arXiv:2605.00073*, 2026.

[3] W3C. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, w3.org/TR/did-core, 2022.

[4] W3C. Verifiable credentials data model v1.1. W3C Recommendation, w3.org/TR/vc-data-model, 2022.

[5] Microsoft. ION: Identity overlay network (Sidetree on Bitcoin). github.com/decentralized-identity/ion, 2020.

[6] Nyx Biderman et al. EIP-5192: Minimal soulbound nfts. Ethereum Improvement Proposal, 2022.

[7] Transient Labs et al. EIP-4973: Account-bound tokens. Ethereum Improvement Proposal, 2022.

[8] Vitalik Buterin, Yoav Forshtat, and et al. EIP-4337: Account abstraction using alt mempool. Ethereum Improvement Proposal, 2021.

[9] Ethereum Attestation Service. Ethereum attestation service (EAS). attest.sh / github.com/ethereum-attestation-service, 2023.

[10] Yulin Liu. A dataset of early blockchain-registered AI agents on ethereum. *arXiv preprint arXiv:2604.22652*, 2026.

[11] Clément Lesaege, Federico Ast, and William George. Kleros: A decentralized court of justice for disputes. Kleros whitepaper, kleros.io, 2018.

[12] Aragon Project. Aragon court: Decentralized arbitration. Aragon docs, aragon.org/dish, 2020.

[13] UMA Protocol. Optimistic oracle: Uma's data verification mechanism. UMA whitepaper, umaproject.org, 2020.

[14] Nexus Mutual. A peer-to-peer discretionary mutual. Nexus Mutual whitepaper, nexusmutual.io, 2019.

[15] Liehuang Zhu. Blockchain empowered trustworthy agent networks. *arXiv preprint arXiv:2608.04626*, 2026.

[16] Anonymous. Zero-trust identity for agentic AI. Working paper (PDF in papers/2505-zero-trust-identity-agentic-ai.pdf), 2025. Metadata to be finalized before submission.

[17] Anonymous. Insured agents. Working paper (PDF in papers/2512-insured-agents.pdf), 2025. Metadata to be finalized before submission.

[18] Anonymous. On binding agent identities. Working paper (PDF in papers/2512-binding-agent-id.pdf), 2025. Metadata to be finalized before submission.

[19] Trustless Agents Working Group. ERC-8004: Trustless agents. Ethereum Request for Comment, 2025.

[20] John R. Douceur. The Sybil attack. In *Peer-to-Peer Systems (IPTPS)*, pages 251–260. Springer, 2002.

[21] Marquis de Condorcet. Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix. *Paris: De l'imprimerie royale*, 1785.

[22] Thomas C. Schelling. *The Strategy of Conflict*. Harvard University Press, 1960.

[23] Vitalik Buterin. An introduction to Truthcoin. Ethereum Foundation Blog, blog.ethereum.org/2015/11/28/truthcoin-and-schelling-points, 2015.

[24] Paul Sztorc. Truthcoin: Peer-to-peer oracle system and prediction markets. Whitepaper, truthcoin.info, 2015.